ChatGPT

Nexus App Design Document

Version: 1.0 (draft)
Date: 2026-02-22 (America/Los_Angeles)
Author: (project team)

Executive summary

This document consolidates the Nexus action-architecture project into an implementation-ready spec for a Discord-first v0 (Railway-hosted bot + SQLite), with a clean path to a web/mesh app and eventually decentralized mirroring and governance.

The core design choice is **a packet protocol**: every Template, MissionPlan, MissionReport, Module, Initiative, Policy, Proposal, Vote, and DecisionRecord is represented as a **canonical JSON "packet"** with (1) a strict envelope/header and (2) a typed body. Packets are hashable via a standard canonicalization algorithm (JCS / RFC 8785) and content-addressable over time, enabling future mirroring and fork/merge semantics that feel like "data packets in git." [1]

For v0 persistence, use SQLite with a "JSON-first, index what you query" pattern: store packet JSON as TEXT, and add **generated columns** and/or **expression indexes** for the high-value query paths. SQLite's generated columns and expression indexes are first-class features, and JSON functions are built-in (JSON1). [2]

Discord UX v0 stays aligned with your current embed-based prototype: **legacy embeds + legacy components** (buttons/selects/modals) rather than Components V2, because Components V2 disables embeds/content for that message when the `IS_COMPONENTS_V2` flag is set. [3]

Operationally, Railway deployments have **ephemeral storage** by default, and persistent state needs a **mounted Volume**; Railway documents both the ephemeral storage behavior and how volumes mount at a service path. [4]

Architecture and data model

System goals

The target system is a **multi-surface client** (Discord bot + web/mesh app) over a **shared packet dataset**:

- Discord is the highest-leverage capture surface for v0 (fast entry, structured forms, immediate social coordination).
- SQLite is the local canonical store for v0 (queryable, migration-friendly, low-ops).
- The web/mesh app becomes a "dataset browser + editor" and later a mirroring node.

This structure anticipates a shift from “Discord as the database” to “Discord as one UI surface,” by explicitly storing Discord message/thread references as *derived* metadata rather than the primary record.

Deployment baseline: v0 on Railway + SQLite

Railway services run as containers, and every deployment has only ephemeral storage by default (with explicit limits). A service that needs persistent state must use a Railway Volume mounted at a path in the container filesystem. [4]

Implication for v0: put `nexus.db` (and any attachment cache) on the mounted volume path, and treat everything else as rebuildable.

High-level architecture diagram

```
flowchart LR
  subgraph Discord
    U[User] -->|Slash cmd / buttons / modals| D[Discord Client]
  end

  D -->|Interactions/Webhook| B[Bot Service]

  subgraph v0 Core
    B --> S[(SQLite)]
    B --> R[Renderer: embeds + components]
  end

  subgraph Future Web/Mesh
    W[Web/Mesh App] --> A[API: packet query + export/import]
    A --> S
  end

  subgraph Future Mirroring
    X[Export Bundle .nxz] --> M[Mirror transports]
    M --> IPFS[IPFS / CAR]
    M --> BT[BitTorrent / DHT pointers]
  end

  S --> X
  IPFS --> W
  BT --> W
```

Core object types and relationships

Nexus revolves around two relationship classes:

1. **Structural references** (typed links): e.g., a MissionPlan “uses” a Template; a MissionReport “reports_on” a MissionPlan; a Policy “governs” an Initiative.
2. **Lineage references** (fork/version): e.g., `fork_of`, `supersedes`, `derived_from`.

These are stored as first-class link edges (not inferred by text scraping).

Packet protocol and versioning

Canonical packet structure

A **packet** is a JSON object with two top-level keys:

- `header`: stable envelope fields used for identity, versioning, provenance, integrity, and compatibility
- `body`: the typed object

Design constraints:

- Packets must be canonicalizable and hashable (for mirroring and integrity).
- Packets must be forward-compatible (capability negotiation and protocol versioning).
- Packets must be linkable (IDs and link objects).

Packet header schema (normative)

Key fields:

- `packet_id` (string): unique immutable packet identifier (recommend ULID for v0)
- `object_id` (string): stable logical identity (“this template across versions”)
- `object_type` (enum): `template | mission_plan | mission_report | module | initiative | policy | proposal | vote | decision_record`
- `schema_version` (string): per-object schema version (SemVer string)
- `packet_version` (integer): monotonically increasing for packets within an object lineage (or SemVer if you want “breaking packet changes”)
- `protocol_version` (string): Nexus protocol version supported by producer
- `app_version` (string): bot/app build version that produced the packet
- `created_at` (ISO-8601 string)
- `supersedes` (optional string packet_id): pointer to previous packet
- `links` (array): typed edges to other objects/packets
- `provenance`: attribution + origin refs (Discord and later web/mesh)
- `integrity`: canonicalization + hashing + optional signatures
- `capabilities`: feature flags and negotiation ranges

Canonicalization requirement: JSON Canonicalization Scheme (JCS) is explicitly designed to produce a deterministic, "hashable" JSON representation for cryptographic operations. [1]

ID conventions

You asked to choose between `nx:type/slug` and ULID-style IDs. The most robust pattern is **dual identifiers**:

- **Stable object identity** (`object_id`): human-friendly, "name-like"
  Format: `nx:<type>/<slug>`
  Example: `nx:mission_plan/trash-pickup`
  This is what people copy/paste, fork, and browse.
- **Immutable packet identity** (`packet_id`): ULID
  ULID is lexicographically sortable (timestamp + randomness), URL-safe, and standardized by an openly published spec (commonly used as a "sortable UUID-like" identifier). [5]

You can also support an alternative `object_id` scheme later (e.g., `nx:<type>/<ulid>`) without breaking the packet protocol, as long as `object_id` remains globally unique.

Versioning layers and strategy

Use **four distinct version layers** (separating concerns so upgrades don't cascade):

Layer	Example	What it versions	Upgrade behavior
`protocol_version`	`1.0.0`	Packet envelope semantics, required fields, link taxonomy	Capability negotiation; reject if incompatible
`schema_version`	`mission_plan@1.1.0`	Body JSON schema for a specific `object_type`	Can be migrated or interpreted with adapters
`packet_version`	`7`	Edits to a specific object over time	Linear chain via `supersedes`
`app_version`	`bot@0.3.2`	Producer implementation	Informational; used for debugging and rollback

Capability negotiation: every producer includes:

- `protocol_version`
- `capabilities.supported_protocols` : `[min, max]`
- `capabilities.supported_schemas` : map of `{object_type: [min, max]}`

Older nodes can still accept newer packets if compatible ranges overlap; otherwise they can store-and-ignore.

Canonicalization, hashing, and future signatures

Hashing baseline: JCS + SHA-256

Process:

1. Build packet JSON object.
2. Normalize using JCS (RFC 8785).
3. Compute `sha256(canonical_bytes)`.
4. Store digest in `header.integrity.digest`.

JCS explicitly targets invariant representations suitable for hashing/signing, using deterministic property sorting and I-JSON constraints. [1]

JSON-LD / URDNA2015 option (future)

For governance and identity proofs, you may later want **graph canonicalization** so that semantically equivalent graphs hash identically. The W3C "RDF Dataset Canonicalization" recommendation standardizes dataset canonicalization (the modern successor to "URDNA2015"-style normalization). [6]

W3C Data Integrity specifications explicitly describe canonicalizing documents using algorithms like URDNA2015 (linked data proofs lineage). [7]

Practical stance:
- v0–v2: JCS for packet integrity and mirroring
- v3+: add optional RDF canonicalization for identity/proof payloads and governance records (where semantics-first hashing matters more than byte-for-byte JSON shape)

Example packet (MissionPlan)

```
{
  "header": {
    "packet_id": "01J2FZ6K8T4K0WQ2W3WQH6S1Q7",
    "object_id": "nx:mission_plan/trash-pickup",
    "object_type": "mission_plan",
    "schema_version": "1.0.0",
    "packet_version": 1,
    "protocol_version": "1.0.0",
    "app_version": "bot@0.1.0",
    "created_at": "2026-02-14T23:45:00Z",
    "supersedes": null,
    "links": [
      {"rel": "uses", "target_object_id": "nx:template/generic-mission"},
      {"rel": "within", "target_object_id": "nx:initiative/keeping-earth-
beautiful"}
    ],
    "provenance": {
```

```
      "created_by": {
        "element_id": "nx:element/uprising-underground",
        "discord_user_id": "123456789012345678",
        "persona_id": null
      },
      "origin": {
        "surface": "discord",
        "guild_id": "234567890123456789",
        "channel_id": "345678901234567890",
        "thread_id": "456789012345678901",
        "message_id": "567890123456789012"
      }
    },
    "integrity": {
      "canonicalization": "RFC8785",
      "hash_alg": "sha-256",
      "digest": "base64url:..."
    },
    "capabilities": {
      "supported_protocols": ["1.0.0", "1.0.0"],
      "attachments": ["discord_cdn_ref"]
    }
  },
  "body": {
    "title": "Trash Pickup",
    "initiative_id": "nx:initiative/keeping-earth-beautiful",
    "template_id": "nx:template/generic-mission",
    "scope": "solo",
    "alignment": {"mode": "open"},
    "participation": {"mode": "integrated"},
    "objectives": [
      "Remove litter from a defined outdoor area",
      "Leave the space measurably cleaner",
      "Interact with the community in positive ways"
    ],
    "coordinators": [{"display_name": "the_uprising"}],
    "schedule": {
      "location_name": "Nature Trail",
      "start_local": "2026-02-14T16:00:00-08:00",
      "duration_minutes": 30
    },
    "modules": {
      "safety": {"items": []},
      "comms": {"channel": "Discord"},
      "supplies": {"items": ["Trash bag"]}
    },
    "status": "planned",
    "visibility": "public"
```

```
  }
}
```

SQLite persistence and query patterns

Why SQLite (v0)

SQLite can serve as the **packet store + query index** with minimal setup, while still supporting future evolution:

- JSON functions/operators are built-in (JSON1) and deterministic. [8]
- You can index "paths into JSON" using either:
- **expression indexes** (index on `json_extract(...)`) [9]
- **generated columns** (virtual columns computed from JSON that you can index) [10]

This gives you document-style flexibility without giving up query performance.

Table design

You requested: `objects`, `object_links`, `discord_refs`, `attachments` (plus migration snippets). I'm including one additional table (`object_packets`) because multi-version packets become painful without it.

objects (object head; mutable)

Stores the latest known state for an object.

- `object_id TEXT PRIMARY KEY`
- `object_type TEXT NOT NULL`
- `slug TEXT`
- `latest_packet_id TEXT NOT NULL`
- `latest_packet_hash TEXT NOT NULL`
- `created_at TEXT NOT NULL`
- `updated_at TEXT NOT NULL`
- `status TEXT NOT NULL`
- `visibility TEXT NOT NULL`
- `participation_mode TEXT`

object_packets (immutable packet history)

Stores all packets (append-only).

- `packet_id TEXT PRIMARY KEY`
- `object_id TEXT NOT NULL`
- `object_type TEXT NOT NULL`
- `schema_version TEXT NOT NULL`

- `packet_version INTEGER NOT NULL`
- `protocol_version TEXT NOT NULL`
- `app_version TEXT NOT NULL`
- `created_at TEXT NOT NULL`
- `supersedes TEXT`
- `canonical_hash TEXT NOT NULL`
- `packet_json TEXT NOT NULL` (full packet)
- `body_json TEXT NOT NULL` (redundant but speeds queries)

object_links (typed edges)

Each row is a directed link between object IDs (or packet IDs if you want).

- `src_object_id TEXT NOT NULL`
- `rel TEXT NOT NULL`
- `dst_object_id TEXT NOT NULL`
- `created_at TEXT NOT NULL`
- composite PK: (`src_object_id`, `rel`, `dst_object_id`)

discord_refs (surface references)

Stores where content lives in Discord (threads/messages/attachments).

Discord IDs are snowflakes across interaction payload structures. [11]

- `object_id TEXT NOT NULL`
- `guild_id TEXT`
- `channel_id TEXT`
- `thread_id TEXT`
- `message_id TEXT`
- `ref_type TEXT` (e.g., `plan_thread`, `report_message`, `template_post`)
- `created_at TEXT NOT NULL`

attachments (evidence abstraction)

Supports “Discord-first v0” then “portable blobs later.”

Discord attachment objects explicitly have an `ephemeral` boolean flag, and ephemeral attachments are guaranteed available only as long as the message exists. [12]

Discord attachment CDN URLs include signed query parameters with expiry, but Discord will refresh attachments when rendering in-client; for embeds/webhooks you can pass the CDN URL without parameters. [13]

- `attachment_id TEXT PRIMARY KEY` (internal)
- `object_id TEXT NOT NULL`
- `source TEXT NOT NULL` (`discord`, `file`, `ipfs`)

- `media_type TEXT`
- `sha256 TEXT` (nullable in v0)
- `byte_length INTEGER`
- `discord_channel_id TEXT`
- `discord_message_id TEXT`
- `discord_attachment_snowflake TEXT`
- `url TEXT` (store CDN base URL **without** expiring params)
- `created_at TEXT NOT NULL`

Schema migration snippet (SQL)

```sql
-- v0.1.0 init
PRAGMA foreign_keys = ON;

-- Recommended for concurrent readers + single-writer workloads:
PRAGMA journal_mode = WAL;

-- Tables
CREATE TABLE IF NOT EXISTS objects (
  object_id TEXT PRIMARY KEY,
  object_type TEXT NOT NULL,
  slug TEXT,
  latest_packet_id TEXT NOT NULL,
  latest_packet_hash TEXT NOT NULL,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  status TEXT NOT NULL,
  visibility TEXT NOT NULL,
  participation_mode TEXT
);

CREATE TABLE IF NOT EXISTS object_packets (
  packet_id TEXT PRIMARY KEY,
  object_id TEXT NOT NULL,
  object_type TEXT NOT NULL,
  schema_version TEXT NOT NULL,
  packet_version INTEGER NOT NULL,
  protocol_version TEXT NOT NULL,
  app_version TEXT NOT NULL,
  created_at TEXT NOT NULL,
  supersedes TEXT,
  canonical_hash TEXT NOT NULL,
  packet_json TEXT NOT NULL,
  body_json TEXT NOT NULL,
  FOREIGN KEY(object_id) REFERENCES objects(object_id)
);
```

```sql
CREATE TABLE IF NOT EXISTS object_links (
  src_object_id TEXT NOT NULL,
  rel TEXT NOT NULL,
  dst_object_id TEXT NOT NULL,
  created_at TEXT NOT NULL,
  PRIMARY KEY (src_object_id, rel, dst_object_id),
  FOREIGN KEY(src_object_id) REFERENCES objects(object_id),
  FOREIGN KEY(dst_object_id) REFERENCES objects(object_id)
);

CREATE TABLE IF NOT EXISTS discord_refs (
  object_id TEXT NOT NULL,
  ref_type TEXT NOT NULL,
  guild_id TEXT,
  channel_id TEXT,
  thread_id TEXT,
  message_id TEXT,
  created_at TEXT NOT NULL,
  PRIMARY KEY (object_id, ref_type, COALESCE(message_id, '')),
  FOREIGN KEY(object_id) REFERENCES objects(object_id)
);

CREATE TABLE IF NOT EXISTS attachments (
  attachment_id TEXT PRIMARY KEY,
  object_id TEXT NOT NULL,
  source TEXT NOT NULL,
  media_type TEXT,
  sha256 TEXT,
  byte_length INTEGER,
  url TEXT,
  discord_channel_id TEXT,
  discord_message_id TEXT,
  discord_attachment_snowflake TEXT,
  created_at TEXT NOT NULL,
  FOREIGN KEY(object_id) REFERENCES objects(object_id)
);
```

Why WAL: WAL mode supports simultaneous readers and writers by appending writes to a separate WAL file and checkpointing back to the main DB. [14]
Operational note: WAL state includes companion files (`-wal`, `-shm`) as part of database state during recovery scenarios, so if you copy the database you need to copy associated hot journal/WAL state as well. [15]

Indexing strategy and high-value query patterns

SQLite supports indexing expressions directly, but the expression must match exactly as written for the planner to use it (no algebraic rewrites). [9]

Generated columns can participate in indexes; VIRTUAL columns can be added later via `ALTER TABLE ADD COLUMN`, while STORED columns cannot be added via ALTER TABLE. [10]

Recommended "index what you query" generated columns

Example: for MissionPlans and MissionReports, you will query `initiative_id`, `template_id`, and `mission_plan_id` constantly.

```
-- Add query columns; VIRTUAL allows adding later.
ALTER TABLE object_packets
ADD COLUMN initiative_id TEXT
GENERATED ALWAYS AS (json_extract(body_json, '$.initiative_id')) VIRTUAL;

ALTER TABLE object_packets
ADD COLUMN template_id TEXT
GENERATED ALWAYS AS (json_extract(body_json, '$.template_id')) VIRTUAL;

ALTER TABLE object_packets
ADD COLUMN mission_plan_id TEXT
GENERATED ALWAYS AS (json_extract(body_json, '$.mission_plan_id')) VIRTUAL;

CREATE INDEX IF NOT EXISTS idx_packets_type_created
ON object_packets(object_type, created_at);

CREATE INDEX IF NOT EXISTS idx_packets_initiative
ON object_packets(initiative_id);

CREATE INDEX IF NOT EXISTS idx_packets_template
ON object_packets(template_id);

CREATE INDEX IF NOT EXISTS idx_packets_plan_reports
ON object_packets(mission_plan_id)
WHERE object_type = 'mission_report';
```

These columns rely on JSON functions; SQLite's JSON support includes `json_extract` and related tools. [8]

Canonical "common queries" (v0)

Fetch a MissionPlan's latest packet + all associated MissionReports:

```
-- Latest plan packet
SELECT p.*
FROM objects o
JOIN object_packets p ON p.packet_id = o.latest_packet_id
```

```
WHERE o.object_id = :plan_object_id;

-- Reports (latest packet per report object_id)
SELECT o.object_id, p.*
FROM objects o
JOIN object_links l ON l.src_object_id = o.object_id
JOIN object_packets p ON p.packet_id = o.latest_packet_id
WHERE l.rel = 'reports_on'
  AND l.dst_object_id = :plan_object_id
ORDER BY p.created_at DESC;
```

List “active” initiatives by recent reports:

```
SELECT initiative_id,
       COUNT(*) AS reports_30d
FROM object_packets
WHERE object_type = 'mission_report'
  AND datetime(created_at) >= datetime('now', '-30 days')
GROUP BY initiative_id
ORDER BY reports_30d DESC;
```

Storage/index options comparison

Option	Pros	Cons	Fit
JSON files on disk (scan/ grep)	simplest mental model	slow queries, hard linking/integrity, painful migrations	prototype only
SQLite JSON store + indexes	fast enough, low ops, supports JSON functions + indexes + generated cols [16]	single-writer constraint; needs schema discipline	**recommended v0–v1**
Postgres JSONB	scalable concurrency, rich indexing	ops overhead, migrations heavier	v2+ if growth demands

Discord UX and permissions

Core Discord primitives you’ll rely on

- **Interactions**: slash commands, message components, and modal submits all arrive as interactions. [11]
- **Ephemeral previews**: Discord allows ephemeral followups via message flag `EPHEMERAL` (`1 << 6`, 64). [11]
- **Modals**: modals are single-user popups for form-like input and can only be opened in response to a command or component interaction. [17]

- **Embed constraints**: embeds have strict per-field limits and an overall 6000-character aggregate limit across embed text fields. [12]
- **Threads in forums**: Discord exposes "Start Thread in Forum or Media Channel" as an endpoint that creates a thread and sends a message inside it. [18]

Components V2 vs your embed-based prototype

Your current prototype is embed-based, and you should keep it that way in v0.

Discord's newer Components V2 mode (`IS_COMPONENTS_V2`) explicitly disables `content` and `embeds` on that message and changes how attachments are displayed. [19]

Discord also documents that **legacy message components** (action rows + buttons/selects alongside embeds/content) still work, with limits like up to 5 action rows at top level. [20]

Decision: v0 uses: - embeds for structured display (matching your screenshots) - legacy action rows for buttons/selects - modals for data entry

Forum layout recommendation for "plan + report in same thread"

You observed a real Discord constraint: forum "posts are threads," so nesting threads under forum posts isn't a clean pattern.

Recommendation for v0:

- **Templates Forum**: each Template is one forum post (thread). Replies contain discussion + forks.
- **Missions Forum**: each MissionPlan is one forum post (thread). MissionReports are **messages in that plan thread**, not separate posts.

This keeps plan/report tightly coupled while you still store strict links in SQLite for cross-browsing later.

Slash commands, modals, and button flows

Discord application commands have naming rules and a defined structure; keep your command tree shallow and stable. [21]

Recommended command set (v0)

Command	Purpose	Notes
`/nx template new`	create a Template	opens modal, then preview
`/nx plan new`	create a MissionPlan	choose Template + fill plan modal
`/nx report submit`	submit a MissionReport	must be invoked *from a plan thread* (or with plan selector)

Command	Purpose	Notes
`/nx link`	link objects (manual fix-up)	admin/curator use
`/nx view`	show object summary	ephemeral by default
`/nx export`	export `.nxz` bundle	admin use initially

Modal field definitions (matching your screenshots)

Keep modal inputs short and structured; store the full arrays in packet JSON; render them compactly in embeds to stay under limits. [12]

MissionPlan modal (v0):

Field (custom_id)	Type	Required	Mapping
`initiative_id`	short text	optional	`body.initiative_id`
`title`	short text	yes	`body.title`
`scope`	select (solo/team/node/multi)	yes	`body.scope`
`alignment_mode`	select (open/invite-only)	yes	`body.alignment.mode`
`participation_mode`	select (integrated/independent/hybrid)	yes	`body.participation.mode`
`location_name`	short text	optional	`body.schedule.location_name`
`start_local`	short text	optional	ISO-8601 local string
`duration_minutes`	short text	optional	integer parse
`objectives`	paragraph text	yes	split lines → array
`coordinators`	short text	optional	split `,` → array
`safety_items`	paragraph text	optional	split lines → array
`comms_channel`	short text	optional	`body.modules.comms.channel`
`supplies`	paragraph text	optional	split lines → array

MissionReport modal (v0):

Field	Type	Required	Mapping
`completion_checklist`	paragraph	yes	line split → checklist items

Field	Type	Required	Mapping
`notes`	paragraph	optional	narrative
`improvements`	paragraph	optional	improvement list
`evidence`	attachment prompt	optional	follow-up message allowing uploads

Embed templates (v0)

All embed layouts must respect embed field limits and total size. [12]

MissionPlan embed template (example)

- **Title:** `<Plan Title> — Mission Plan`
- **Description:** (optional) short tagline + template link
- **Fields:**
- Initiative
- Version (schema/body)
- Scope
- Alignment
- Participation
- Coordinators
- Location / Date / Time (inline)
- Duration
- Objectives (as numbered lines)
- Safety Protocol(s)
- Communications
- Supply Checklist
- **Footer:** `Nexus — <Element Name>`

MissionReport embed template (example)

- **Title:** `Report for mission: <Plan Title>`
- **Description:** link back to plan thread/message
- **Fields:**
- Initiative / Version / Scope
- Participant (provenance display)
- Location / Date / Time / Duration
- “Were the following objectives completed?” checklist render
- Notes
- Improvement Opportunities
- Attachments summary (count + types)

Minimal permission model (v0)

Use Discord’s existing guild/channel permissions + application command permissions where possible. Discord documents both guild permission concepts and application command permissions. [22]

A minimal workable policy:

Action	Default	Escalation path
Create MissionReport	anyone with SEND_MESSAGES in mission thread	none
Create MissionPlan	anyone (but mark as “unverified”)	“Coordinator” role can mark verified
Create/Publish Template	restricted	“Curator” role
Edit existing Template/ Policy	restricted	“Curator” role
Export bundles	restricted	admin/maintainers

Keep the permission model small, and push trust signals into **badges** and **provenance** rather than trying to pre-moderate everything.

Mirroring, export bundles, and roadmap

Export/import bundle format (`.nxz`)

Define `.nxz` as a deterministic archive with a manifest:

```
bundle.nxz (tar + compression)
  /manifest.json
  /packets/<packet_id>.json
  /blobs/<sha256>.<ext>          (optional v1+)
  /indexes/links.csv             (optional)
```

`manifest.json` includes:

- `bundle_version`
- `created_at`
- `protocol_version_range`
- `root_objects` : list of root `object_id` s included
- `packet_digests` : map packet_id → sha256
- `blob_digests` : map sha256 → metadata
- `beacons` : optional pointers (see below)

Beacon/pointer model (for “torrent-like mirrors”)

Your long-term mirroring idea needs a way to publish “where the newest dataset head is” without central servers.

Two primary-source-aligned options:

- **IPNS pointer**: IPNS is explicitly a mutable link system for IPFS content, allowing publishing "latest version" even as underlying hashes change. [23]
- **BitTorrent DHT items**: BEP 44 defines storing arbitrary data in the BitTorrent DHT, including immutable items (keyed by SHA-1 of the data) and mutable items (keyed by a public key and signed). [24]

Nexus pointer packet (future):
A `pointer` object that says: "latest bundle digest is X; retrieval hints are {ipfs_cid, magnet_uri, http_gateway_urls}."

Canonical content-addressing alignment with IPFS

IPFS content identifiers (CIDs) are explicitly based on a cryptographic hash of content; changes in content yield different CIDs, and identical content yields identical CIDs under the same settings. [25]

IPFS also defines CAR files (Content Addressable aRchives) as a way to package content-addressed blocks for storage/transfer, with CAR v1/v2 variants. [23]

Forward plan: once packets/blobs are content-addressed, you can: - export `.nxz` for human/tool workflows - optionally emit a CAR representation for IPFS-native transport

Health, maturity, and activity badges (heuristics + formulas)

Your earlier design mentions (1) age, (2) size/contents, (3) completed mission reports, plus "recent vs lifetime."

Define three computed metrics per object `0` (initiative/template/plan/module):

Let:

- `age_days = now - first_seen_at`
- `active_days = now - last_activity_at`
- `reports_30d`, `reports_all`
- `plans_30d`, `plans_all`
- `forks_all`
- `completions_30d` = count of reports marked completed (or objective completion rate)

Activity score (A):

```
A = ln(1 + 3*reports_30d + 1*plans_30d + 0.5*forks_30d)
```

Maturity score (M):

```
M = min(1, ln(1 + reports_all) / ln(1 + 25))
```

(25 reports ≈ “fully field-tested” maturity)

Health score (H):

```
H = 0.5 * clamp01(1 - active_days/90)
  + 0.5 * clamp01(objective_completion_rate_30d)
```

Badge mapping:

- Activity: `A >= 2.0` → ; `A >= 1.0` → ; `A >= 0.3` → ; else ○
- Maturity: `M >= 0.8` → ; `M >= 0.4` → ; else
- Health: `H >= 0.7` → ; `H >= 0.4` → ⚠; else

These are intentionally simple and tunable; the key is that all inputs are queryable from `object_packets` + `object_links`.

Participation modes: integrated / independent / hybrid

Add a single field (required on MissionPlan and optionally Initiative):

```
"participation": {
  "mode": "integrated | independent | hybrid",
  "integrated_contract": {
    "join_coordination": true,
    "voting_rights": true
  },
  "independent_guidance": {
    "coordination_channel": "discord",
    "recommended_checkin": "optional"
  }
}
```

UI affordances (Discord): - Plan embed shows Participation badge: `INTEGRATED`, `INDEPENDENT`, `HYBRID` - Button row offers: - `Join (integrated)` → optional roster intent (even if anonymous later) - `Align (independent)` → “I’m contributing independently” marker - Reports can optionally declare participation mode at time of report submission.

Provenance and attribution model

In v0, provenance should be explicit but lightweight:

- `element_id`: group identity (server/community-level identity)
- `discord_user_id`: concrete attribution in Discord
- `persona_id`: nullable placeholder for future pseudonymous cryptographic identity

Interaction payloads include invoking user/member context and snowflake IDs, which you can store in provenance + `discord_refs`. [11]

Attachments/evidence strategy

Discord-first v0: - store Discord message IDs and attachment references - store CDN URLs without expiring query params (Discord will render/refresh when used in-client) [13] - treat ephemeral attachments as non-durable evidence unless you also ingest/copy them. [12]

Portable evidence v1–v2: - optionally download and hash blobs → store in `attachments.sha256` - include blobs in `.nxz` bundles - optionally publish blobs as IPFS blocks / CAR archives later. [26]

Phased roadmap (v0–v3)

v0: Discord-first bot + SQLite (Railway) - Implement packet envelope + schemas for: Template, MissionPlan, MissionReport, Module, Initiative - Implement SQLite schema + indexes + migrations (WAL mode) - Implement Discord flows: `/nx template new`, `/nx plan new`, `/nx report submit` with previews - Enforce embed constraints, store `discord_refs`, compute basic badges - Use Railway Volume for persistence (DB + optional cache) [4]

v1: Web app mirror of the canonical dataset - Read-only web explorer for objects/links/badges - Export/import `.nxz` bundles + manifest verification (JCS+SHA-256) [1] - Add `object_packets` history browsing and fork visualization

v2: Decentralized mirroring - Add pointer packets (“dataset beacons”) - Publish pointers via IPNS and/or BitTorrent DHT items (BEP 44) [27] - Add optional CAR export for IPFS-native transport [23]

v3: Governance packets - Introduce Policy, Proposal, Vote, DecisionRecord schemas - Implement vote casting + tally + decision records - Consider graph-canonicalization/signed proofs for governance objects (W3C RDF canonicalization / Data Integrity lineage) [28]

Prioritized next steps

1. Lock the **packet envelope** fields and ID strategy (object_id vs packet_id) and implement JCS hashing (`header.integrity`). [29]
2. Implement SQLite schema + 3–5 critical indexes (initiative_id, template_id, mission_plan_id, object_type+created_at). [2]
3. Build v0 Discord flow end-to-end:
   `/nx plan new → modal → ephemeral preview → post plan thread` and `/nx report`

`submit → modal → post report message in plan thread`. (Keep legacy embeds/components.) [30]

4. Add badge computation in SQLite queries (materialize later if needed).
5. Add export `.nxz` with manifest and packet digests; import should validate digests before writing.

Open questions

- Do you want **object_id slugs** to be globally unique, or only unique within an element/namespace (e.g., `nx:<element>/<type>/<slug>`)?
- For MissionPlans: is "scope" best modeled as a single enum (`solo/team/node/multi`) or a set (`{solo, open, multi_scope}`) like your template screenshot?
- What is the minimum governance slice you want *inside this v1.0 spec* (just schemas, or also v3 UX flows)?
- Should Template be a "pure schema" (fields/modules only), or should it also carry recommended default values and render rules (embed layout constraints)?
- For evidence: do you want v0 to immediately **ingest and hash** image attachments (more durable), or stay "Discord-reference-only" until v1? [31]

---

[1] [29] https://www.rfc-editor.org/rfc/rfc8785
https://www.rfc-editor.org/rfc/rfc8785

[2] [10] Generated Columns
https://sqlite.org/gencol.html

[3] [19] [20] https://docs.discord.com/developers/docs/interactions/message-components%2A
https://docs.discord.com/developers/docs/interactions/message-components%2A

[4] https://docs.railway.app/develop/services
https://docs.railway.app/develop/services

[5] https://github.com/ulid/spec
https://github.com/ulid/spec

[6] [28] https://www.w3.org/TR/rdf-canon/
https://www.w3.org/TR/rdf-canon/

[7] Data Integrity 1.0
https://www.w3.org/community/reports/credentials/CG-FINAL-data-integrity-20220722/?utm_source=chatgpt.com

[8] [16] JSON Functions And Operators
https://sqlite.org/json1.html

[9] Indexes On Expressions
https://sqlite.org/expridx.html

[11] [30] Interactions - Documentation - Discord
https://docs.discord.com/developers/interactions/receiving-and-responding

[12] [31] https://docs.discord.com/developers/resources/message
https://docs.discord.com/developers/resources/message

13 https://docs.discord.com/developers/reference
https://docs.discord.com/developers/reference

14 https://sqlite.org/wal.html
https://sqlite.org/wal.html

15 https://sqlite.org/fileformat.html
https://sqlite.org/fileformat.html

17 Overview of Interactions - Documentation
https://docs.discord.com/developers/interactions/overview?utm_source=chatgpt.com

18 https://docs.discord.com/developers/resources/channel
https://docs.discord.com/developers/resources/channel

21 https://docs.discord.com/developers/interactions/application-commands
https://docs.discord.com/developers/interactions/application-commands

22 Permissions - Documentation
https://docs.discord.com/developers/topics/permissions?utm_source=chatgpt.com

23 26 27 https://docs.ipfs.tech/concepts/glossary/
https://docs.ipfs.tech/concepts/glossary/

24 https://www.bittorrent.org/beps/bep_0044.html
https://www.bittorrent.org/beps/bep_0044.html

25 https://docs.ipfs.tech/concepts/content-addressing/
https://docs.ipfs.tech/concepts/content-addressing/